Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación
y Diseño Digital

Carrera de Software

Fichas bibliográficas: fundamentos de sistemas distribuidos y arquitecturas distribuidas

Bibliographic records: fundamentals of distributed systems and distributed architectures

(FORM-FB-01)

Asignatura:
Aplicaciones Distribuidas (ISR-701)

Autor:
Apunte Pazmiño Washington German

Docente:
Gleiston Guerrero Ulloa, PhD

Nivel y paralelo:
Séptimo nivel, paralelo B

Afiliación:
UTEQ

Modalidad individual · Semana 2 (2 al 8 de noviembre de 2026)
8 de noviembre de 2026

Índice

1. Introducción y criterios de selección **2**

2. Fichas bibliográficas **3**
2.1. Ficha 1: arquitecturas en el continuo borde–nube 3
2.2. Ficha 2: comunicación asíncrona con brokers de mensajes 4
2.3. Ficha 3: consistencia y replicación en bases NoSQL 5
2.4. Ficha 4: tolerancia a fallos e ingeniería del caos 6
2.5. Ficha 5: observabilidad, anomalías y causa raíz 7

3. Uso previsto de las fuentes en TA-IND-01 **8**

4. Declaración de uso de inteligencia artificial **8**

Referencias **8**

Índice de tablas

1. Fuentes seleccionadas 2
2. Uso previsto de cada fuente en TA-IND-01 8

1. Introducción y criterios de selección

Este documento reúne cinco fichas bibliográficas que servirán como base del Informe Académico Individual TA-IND-01, donde se analizará de forma crítica un sistema distribuido real. Las fuentes se eligieron para cubrir, cada una desde un ángulo distinto, los temas que exige ese análisis: la arquitectura del sistema, la forma en que se comunican sus componentes, la consistencia de los datos replicados, la tolerancia a fallos y la observabilidad.

Para seleccionarlas se aplicaron estos criterios:

- **Actualidad:** publicaciones de 2024 a 2026, dentro del periodo exigido (2021–2026); tres de las cinco son de 2025 o 2026.
- **Calidad:** solo artículos de revistas indexadas en Scopus, publicadas por Springer Nature y Elsevier.
- **Acceso abierto:** el PDF de los cinco artículos se descarga gratis desde el sitio del editor, lo que permitió leerlos completos y ubicar la página o sección de cada dato citado.
- **Verificación:** cada DOI se abrió en `https://doi.org` y llevó a la página del editor del artículo citado. Los autores, el año, el título, la revista, el volumen, el número y las páginas o el número de artículo se compararon con esa página y con el registro Crossref del DOI (consulta del 7 de noviembre de 2026).

La Tabla 1 resume las fuentes; las referencias completas en formato IEEE se generan con `biblatex` desde el archivo `referencias.bib`.

Tabla 1: Fuentes seleccionadas

Ref.	Tema	Revista (editorial)	Año	Tipo de estudio
[1]	Arquitecturas en el continuo borde–nube	*Cluster Computing* (Springer)	2025	Revisión sistemática
[2]	Comunicación asíncrona con brokers de mensajes	*Journal of Parallel and Distributed Computing* (Elsevier)	2025	Algoritmo, simulación e implementación
[3]	Consistencia y replicación en bases NoSQL	*Journal of Cloud Computing* (Springer)	2024	Estudio experimental
[4]	Tolerancia a fallos e ingeniería del caos	*Computing* (Springer)	2024	Estudio empírico exploratorio
[5]	Observabilidad: anomalías y causa raíz	*Cluster Computing* (Springer)	2026	Revisión sistemática

2. Fichas bibliográficas

2.1. Ficha 1: arquitecturas en el continuo borde–nube

Referencia (IEEE)	J. Pournazari, A. Ullah, A. Al-Dubai, and X. Liu, "Computation offloading in the edge-to-cloud compute continuum: A survey of federated architectural solutions," *Cluster Computing*, vol. 28, no. 13, 2025, Art. no. 839, doi: 10.1007/s10586-025-05577-6
DOI	https://doi.org/10.1007/s10586-025-05577-6
Tipo de fuente	Artículo de revista: revisión sistemática (PRISMA). *Cluster Computing*, Springer Nature. ISSN 1386-7857, e-ISSN 1573-7543. Indexada en Scopus, Q1 en SJR 2024. Acceso abierto (CC BY 4.0), con PDF gratuito en el sitio del editor.
Resumen	Cuando una aplicación exige baja latencia, maneja datos sensibles o depende de recursos repartidos cerca de los datos, la nube por sí sola se queda corta. Los autores abordan la descarga de cómputo (*computation offloading*) a lo largo del continuo borde–nube. Ponen el foco en las soluciones descentralizadas y federadas, un terreno que las revisiones apenas habían explorado. Siguiendo el protocolo PRISMA sobre publicaciones de 2018 a 2024, partieron de 607 resultados y conservaron 88. Con ellos construyen una taxonomía de cuatro dimensiones (arquitectura, estrategia, objetivos y evaluación) y señalan siete desafíos, entre ellos la movilidad y la tolerancia a fallos.
Dato concreto	Entre las soluciones descentralizadas revisadas, P2P es el patrón de comunicación más adoptado (37.5 %), y Raft y Zab son los protocolos de consenso más usados, con 14.8 % cada uno [1, Sec. 5.3, p. 32]. El artículo también define el continuo borde–nube como la extensión de la nube con dispositivos de bajo consumo y baja latencia ubicados cerca de las fuentes de datos [1, Sec. 1, p. 2].
Nota personal	Hasta ahora pensaba que diseñar un sistema distribuido era, en el fondo, elegir entre cliente-servidor y microservicios. La revisión me hizo ver que en el continuo borde–nube coexisten varios patrones. También aprendí que, cuando no hay un coordinador fijo, los nodos llegan a acuerdos mediante protocolos de consenso como Raft, que elige un líder de forma dinámica. En TA-IND-01 la usaré en la sección de arquitectura, para clasificar el sistema elegido como centralizado, jerárquico o P2P y explicar en qué punto procesa sus datos. La descarga de cómputo entre nodos me servirá además como ejemplo concreto de transparencia de migración.

2.2. Ficha 2: comunicación asíncrona con brokers de mensajes

Referencia (IEEE)	D. Landau, N. Saurabh, X. Andrade, and J. G. Barbosa, "Latency and cost-aware consumer group autoscaling in message broker systems," *Journal of Parallel and Distributed Computing*, vol. 201, 2025, Art. no. 105071, doi: `10.1016/j.jpdc.2025.105071`
DOI	`https://doi.org/10.1016/j.jpdc.2025.105071`
Tipo de fuente	Artículo de revista: propuesta algorítmica con simulación e implementación. *Journal of Parallel and Distributed Computing*, Elsevier. ISSN 0743-7315. Indexada en Scopus. Acceso abierto (CC BY 4.0), con PDF gratuito en ScienceDirect.
Resumen	En un *broker* como Kafka, los consumidores de un grupo se dividen las particiones de un tema para procesarlas en paralelo. Cuántos consumidores mantener y cuándo reasignar particiones son decisiones que pesan en el costo y la latencia, porque cada rebalanceo detiene la lectura. El artículo plantea el problema como un *bin packing* con ítems de tamaño variable. Sobre esa base adapta heurísticas que consideran tanto el costo de operación como el de rebalanceo. En simulación, la variante MWF logró menor latencia que Kafka con la misma cantidad de consumidores, y luego la propuesta se implementó sobre Kafka y Kubernetes.
Dato concreto	En las simulaciones, MWF (*modified worst fit*) alcanzó una latencia de percentil 99 de 2.24 s, mientras que la configuración de Kafka con el mismo costo operativo (15 consumidores) llegó a 364.66 s [2, Sec. 6.4]. En las conclusiones, los autores añaden que Kafka requiere al menos 60 % más consumidores que su método [2, Sec. 9].
Nota personal	Para mí, la comunicación asíncrona era simplemente colocar una cola entre dos servicios. El artículo me mostró que el *broker* divide el trabajo en particiones y que moverlas no es gratis. Mientras una partición pasa de un consumidor a otro, nadie la lee, y eso se nota en la latencia. Lo aprovecharé en dos secciones de TA-IND-01. En modelos de comunicación, me servirá para describir el esquema de publicación/suscripción del sistema elegido. En escalabilidad, lo usaré para analizar cómo debería aumentar el número de consumidores a medida que sube la carga.

2.3. Ficha 3: consistencia y replicación en bases NoSQL

Referencia (IEEE)	S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo, and E. Andrade, "Impacts of data consistency levels in cloud-based NoSQL for data-intensive applications," *Journal of Cloud Computing*, vol. 13, no. 1, 2024, Art. no. 158, doi: `10.1186/s13677-024-00716-7`
DOI	`https://doi.org/10.1186/s13677-024-00716-7`
Tipo de fuente	Artículo de revista: estudio experimental. *Journal of Cloud Computing*, Springer Nature (SpringerOpen). e-ISSN 2192-113X. Indexada en Scopus. Acceso abierto (CC BY-NC-ND 4.0), con PDF gratuito en el sitio del editor.
Resumen	¿Qué precio se paga por exigir consistencia fuerte en una base NoSQL replicada? El trabajo busca medirlo, partiendo de que la latencia pesa más cuanto más separadas están las réplicas. Los autores montaron Cassandra, MongoDB y Redis en Google Cloud con clústeres de tres nodos, dentro de una región y después en varias. Contrastaron niveles de consistencia débiles y fuertes, como ONE y ALL en Cassandra. Las pruebas se hicieron con Locust, de 1 a 10 usuarios concurrentes, en 200 escenarios. Los tres motores mostraron el mismo patrón: a mayor consistencia, menor *throughput* y mayor tiempo de respuesta, sobre todo en el despliegue multirregión.
Dato concreto	Al pasar de consistencia débil a fuerte en el despliegue multirregión, el *throughput* de escritura cayó en promedio 95.12 % y su tiempo de respuesta aumentó 3454.34 %. En una sola región, las variaciones promedio fueron menores: una caída de 32.94 % y un aumento de 78.33 %, respectivamente [3, Tabla 8, p. 14].
Nota personal	Del teorema CAP conocía la definición, pero nunca lo había visto expresado en números. El artículo no habla de PACELC. Aun así, sus resultados encajan con la parte «ELC» de ese modelo: incluso sin particiones de red, optar por consistencia fuerte cuesta latencia, y ese costo aumenta con la distancia entre réplicas. En TA-IND-01 lo ubicaré en la sección de consistencia y replicación (CAP y PACELC). Allí pretendo justificar qué nivel de consistencia requiere cada operación del sistema analizado y qué pasaría si sus datos se replicaran en distintas regiones.

2.4. Ficha 4: tolerancia a fallos e ingeniería del caos

Referencia (IEEE)	A. Al-Said Ahmad, L. F. Al-Qora'n, and A. Zayed, "Exploring the impact of chaos engineering with various user loads on cloud native applications: An exploratory empirical study," *Computing*, vol. 106, no. 7, pp. 2389–2425, 2024, doi: `10.1007/s00607-024-01292-z`
DOI	`https://doi.org/10.1007/s00607-024-01292-z`
Tipo de fuente	Artículo de revista: estudio empírico exploratorio. *Computing*, Springer Nature. ISSN 0010-485X, e-ISSN 1436-5057. Indexada en Scopus. Acceso abierto (CC BY 4.0), con PDF gratuito en el sitio del editor.
Resumen	Los autores examinan cómo reaccionan las aplicaciones nativas de la nube ante retardos inyectados bajo distintas cargas. También evalúan si estos experimentos sacan a la luz fallos de disponibilidad que el análisis de confiabilidad pasa por alto. Aplicaron ingeniería del caos en AWS sobre dos casos. El primero fue una aplicación *serverless* en .NET basada en funciones Lambda, y el segundo, la tienda PrestaShop en contenedores con MySQL. Los retardos fueron de 300 ms a 27.5 s, mientras JMeter simulaba entre 150 y 1200 usuarios. Las dos aplicaciones dejaron ver debilidades y puntos de fallo, como peticiones que nunca se atendieron y caídas del *throughput*.
Dato concreto	Con 1200 usuarios, el *throughput* de PrestaShop cayó, respecto de la línea base, 23 % con retardos de 300 ms y 87 % con retardos de 15 000 ms [4, Sec. 4.2.3, p. 2418]. En la aplicación *serverless*, el 17 % de las solicitudes quedó sin atender en los experimentos con retardo de 10 000 ms [4, Sec. 4.1.3, p. 2411]. Los autores definen la ingeniería del caos como una disciplina que ayuda a identificar y corregir debilidades potenciales en sistemas complejos [4, Sec. 2.2, p. 2394].
Nota personal	Yo relacionaba la tolerancia a fallos únicamente con tener servidores replicados. Este trabajo me hizo ver que también hay que comprobarla, provocando fallos de manera controlada. Con un retardo de 10 s, el 17 % de las solicitudes se quedó sin respuesta. Lo tomaré como base en la sección de tolerancia a fallos de TA-IND-01. Allí propondré un experimento de caos sobre el sistema elegido y fijaré las métricas a observar (latencia, *throughput* y tasa de error) antes de emitir un juicio sobre su disponibilidad.

2.5. Ficha 5: observabilidad, anomalías y causa raíz

Referencia (IEEE)	L. M. Barata, S. Sequeira, E. Lopes, P. R. M. Inácio, and M. M. Freire, "Anomaly detection and root-cause identification in microservices: A survey," *Cluster Computing*, vol. 29, no. 5, 2026, Art. no. 309, doi: `10.1007/s10586-026-06095-9`
DOI	`https://doi.org/10.1007/s10586-026-06095-9`
Tipo de fuente	Artículo de revista: revisión sistemática (PRISMA). *Cluster Computing*, Springer Nature. ISSN 1386-7857, e-ISSN 1573-7543. Indexada en Scopus, Q1 en SJR 2024. Acceso abierto (CC BY 4.0), con PDF gratuito en el sitio del editor.
Resumen	En los microservicios los cambios son frecuentes, conviven tecnologías heterogéneas y cada servicio depende de otros. Por eso cuesta detectar anomalías y rastrear su causa raíz, y muchos métodos requieren escoger métricas a mano o acudir a expertos. Esta revisión sistemática, elaborada con PRISMA, reúne estudios hasta 2025 en bases como IEEE Xplore, ACM, SpringerLink, Scopus y Web of Science. Los autores agrupan los enfoques según la fuente de datos (logs, trazas distribuidas y métricas) y el tipo de aprendizaje. Concluyen que pocos trabajos se ocupan de identificar la causa raíz. Proponen avanzar hacia un diagnóstico distribuido confiable, y no solo preciso.
Dato concreto	Al comparar los métodos, la revisión afirma que «Log-based is the most accurate data collection method» y que la combinación de logs y trazas es la más equilibrada [5, Sec. 5.4, p. 27]. También señala que los métodos basados en monitoreo encabezan la recolección de datos, aunque las técnicas basadas en trazas dominaron en los últimos tres años [5, Sec. 5.2.1, p. 24].
Nota personal	Entendí que la observabilidad no puede separarse de la tolerancia a fallos. En un sistema compuesto por cientos de servicios, si no se generan registros, trazas y métricas que realmente aporten información, resulta imposible identificar dónde se originó un error y cómo se extendió hacia otros componentes. En el TA-IND-01 incluiré esta revisión en dos secciones: en la de tolerancia a fallos, para describir cómo el sistema seleccionado detecta y diagnostica sus errores, y en la postura crítica, para analizar si su monitoreo se apoya en una única fuente de datos o si integra varias.

3. Uso previsto de las fuentes en TA-IND-01

La Tabla 2 ubica cada fuente en la sección del informe TA-IND-01 donde se empleará.

Tabla 2: Uso previsto de cada fuente en TA-IND-01

Sección de TA-IND-01	Fuente	Aporte
Arquitectura del sistema	[1]	Patrones cliente/servidor, jerárquico y P2P; ubicación del procesamiento en el continuo borde–nube.
Modelos de comunicación	[2]	Publicación/suscripción, particiones y grupos de consumidores en un broker.
Consistencia y replicación (CAP y PACELC)	[3]	Costo medido de la consistencia fuerte y compromiso entre latencia y consistencia.
Escalabilidad y transparencias	[1], [2]	Escalado de consumidores; migración de cómputo y reubicación de particiones como ejemplos de transparencia.
Tolerancia a fallos	[4], [5]	Pruebas de resiliencia por inyección de fallos; detección de anomalías y diagnóstico de la causa raíz.
Análisis crítico y postura	[3]–[5]	Evidencia cuantitativa para juzgar las decisiones de diseño del sistema elegido.

4. Declaración de uso de inteligencia artificial

Herramienta	Claude (Anthropic), modelo Claude Opus 5.5, en la aplicación Claude.
Para qué se usó	Para buscar artículos candidatos y revisar sus metadatos en Crossref, comprobar que el PDF de cada uno es de libre acceso, extraer del texto completo cada dato concreto y redactarlo con su página o sección, redactar la introducción y la tabla de uso previsto, generar el código LaTeX, el archivo `.bib` y el `README.md`, y preparar un primer borrador de los resúmenes y las notas personales, que después reescribí con mis propias palabras; la IA solo revisó la ortografía y la gramática de esa versión final y recortó algunas palabras para ajustar los resúmenes a la extensión pedida.
Cómo se verificó	Cada DOI se abrió en `https://doi.org` y llevó al artículo citado. Autores, año, título, revista, volumen, número y páginas o número de artículo se cotejaron con la página del editor y con el registro Crossref. Cada dato concreto se buscó en el texto completo del artículo para confirmar su sección o página. El documento se compiló sin errores y se revisó antes de subirlo al repositorio.

Referencias

[1] J. Pournazari, A. Ullah, A. Al-Dubai, and X. Liu, "Computation offloading in the edge-to-cloud compute continuum: A survey of federated architectural solutions," *Cluster Computing*, vol. 28, no. 13, 2025, Art. no. 839, doi: 10.1007/s10586-025-05577-6.

[2] D. Landau, N. Saurabh, X. Andrade, and J. G. Barbosa, "Latency and cost-aware consumer group autoscaling in message broker systems," *Journal of Parallel and Distributed Computing*, vol. 201, 2025, Art. no. 105071, doi: 10.1016/j.jpdc.2025.105071.

[3] S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo, and E. Andrade, "Impacts of data consistency levels in cloud-based NoSQL for data-intensive applications," *Journal of Cloud Computing*, vol. 13, no. 1, 2024, Art. no. 158, doi: 10.1186/s13677-024-00716-7.

[4] A. Al-Said Ahmad, L. F. Al-Qora'n, and A. Zayed, "Exploring the impact of chaos engineering with various user loads on cloud native applications: An exploratory empirical study," *Computing*, vol. 106, no. 7, pp. 2389–2425, 2024, doi: 10.1007/s00607-024-01292-z.

[5] L. M. Barata, S. Sequeira, E. Lopes, P. R. M. Inácio, and M. M. Freire, "Anomaly detection and root-cause identification in microservices: A survey," *Cluster Computing*, vol. 29, no. 5, 2026, Art. no. 309, doi: 10.1007/s10586-026-06095-9.